

ThunderERP

Frontend Engineering Reference Guide

React + TypeScript + Vite

Intellectual Property of AB Global Traders

Stack: React 18 • TypeScript • Vite • TanStack Query • Zustand • Tailwind CSS

Version 1.0 — Internal Use Only

0. How to Use This Document

This is your permanent, living reference for building production-grade ERP interfaces. You will NOT understand everything the first time — that is completely normal. Treat this document like a technical encyclopedia. Come back every time you build a screen, debug an API call, or review UI code. It is designed to stay useful even after 2–3 years of professional experience.

 **KEY INSIGHT:** Feeling overwhelmed is a sign you are reading the right material. Push through. Every concept that confuses you today will click after you build your first real screen with it.

Layered Reading Plan

- Layer 1 — Days 1–3: Understand the Golden Frontend Flow and complete environment setup.
- Layer 2 — Week 1: Master components, hooks, TypeScript, and basic API integration.
- Layer 3 — Weeks 2–4: Build full ERP modules (Inventory, Sales, CRM) with routing, auth, and state.
- Layer 4 — Month 2+: Advanced patterns, performance optimization, testing, and full backend integration.
- Ongoing: Refer back for debugging playbooks, UI patterns, and contract rules.

 **HABIT TO BUILD:** Every time you finish a screen, open this document and check the Golden Rules (Section 20) and Common Mistakes (Section 22). Make it a ritual.

1. System Overview — Frontend in ThunderERP

The frontend is the interaction layer between the user and the entire ThunderERP system. It is built with React + TypeScript + Vite and connects exclusively to the NestJS backend via REST APIs. Every pixel the user sees, every button they click, every form they submit — all of that lives here.

 **GOLDEN RULE:** The frontend does NOT own business logic. It never calculates tax, validates stock levels, or decides if a discount is valid. It only asks the backend "Is this allowed?" and displays the result. If it involves money, stock, permissions, or ERP rules → it belongs in the NestJS Service.

1.1 Responsibility Separation — Never Break This

Layer	Technology	Responsibility & What It NEVER Does
Frontend	React + TypeScript	UI rendering, user input capture, basic UX validation, state management. NEVER: business rules, financial calculations, database access.
Backend	NestJS + TypeScript	All business logic, validation, security, transactions. NEVER: render UI, manage browser state.
Database	PostgreSQL + Prisma	Data persistence, integrity enforcement. NEVER: any UI or frontend concern.

1.2 The Golden Frontend Flow

Use this flow to debug ANY problem. When something breaks, trace it step by step from top to bottom.

```
User Action (button click / form submit)
  → Component Event Handler (onClick / onSubmit)
  → API Service Call (Axios instance)
  → TanStack Query manages cache and loading state
  → NestJS Backend processes request
  → Response returns with Swagger-defined shape
  → TanStack Query updates its cache
  → React re-renders affected components automatically
  → User sees updated UI
```

 **DEBUGGING RULE:** UI not updating? Start at the top — did the event fire? API service called? Check Network tab — did the request go out? Check response shape — does it match the Swagger contract? Check queryKey — is TanStack Query invalidating correctly?

1.3 Why This Stack?

Technology	Why We Chose It
React 18	Industry standard for complex UIs. Concurrent features, massive ecosystem, hooks-based architecture.
TypeScript	Catches type mismatches between frontend and backend at compile time, not at runtime in production.
Vite	Starts development server in under 1 second. Faster than Create React App. Hot Module Replacement is instant.
TanStack Query	Manages all server state: caching, background refetching, pagination, loading/error states automatically.
Zustand	Minimal global state for auth tokens, user info, and app-wide settings. Simpler than Redux.
Tailwind CSS	Utility-first CSS. No context switching to separate style files. Consistent design system.
React Hook Form + Zod	Performant forms with schema-based validation that mirrors our NestJS DTO validation rules.
shadcn/ui	Pre-built accessible components that we own and customize — not a black-box UI library.

2. Environment Setup

2.1 Install Node.js

React runs on Node.js. You must install it before anything else. Always install the LTS (Long Term Support) version — never the "Current" version.

1. Go to <https://nodejs.org> and download the LTS version (v20 or v22).
2. Run the installer for your operating system (Windows / macOS / Linux).
3. Restart your computer after installation.
4. Verify by opening Terminal (Mac/Linux) or PowerShell (Windows):

```
node -v    # Should show: v20.x.x or v22.x.x
npm -v     # Should show: 10.x.x
```

⚠ NOTE: If you see an error after running `node -v`, the installation failed. Download and reinstall from nodejs.org. Do not proceed until this works.

2.2 Create the React Project

We use Vite instead of the older Create React App. Vite is dramatically faster for development.

```
# Step 1: Create the project
npm create vite@latest thunder-erp-frontend -- --template react-ts

# Step 2: Enter the project folder
cd thunder-erp-frontend

# Step 3: Install dependencies
npm install

# Step 4: Start the development server
npm run dev

# Open browser: http://localhost:5173
# You should see the Vite + React welcome page.
```

2.3 Install Core Dependencies

```
# Stop the dev server first (Ctrl + C), then:

# Core runtime dependencies:
npm install react-router-dom@6
npm install axios
npm install @tanstack/react-query
npm install zustand
npm install react-hook-form
```

```

npm install zod @hookform/resolvers

# UI components:
npm install class-variance-authority clsx tailwind-merge
npm install lucide-react

# Toast notifications:
npm install sonner

# Tailwind CSS (styling framework):
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

# Dev tools:
npm install -D @types/node eslint prettier
npm install -D @tanstack/react-query-devtools
npm install -D vitest @testing-library/react @testing-library/user-event jsdom

```

2.4 Configure Tailwind CSS

```

// tailwind.config.js
export default {
  content: ["/index.html", "./src/**/*..{js,ts,jsx,tsx}"],
  theme: {
    extend: {
      colors: {
        primary: { DEFAULT: "#1E3A5F", ... },
        accent: { DEFAULT: "#E8480C", ... },
        success: { DEFAULT: "#2A7F62", ... },
      }
    }
  },
  plugins: [],
}

// src/index.css — add at top:
@tailwind base;
@tailwind components;
@tailwind utilities;

```

2.5 Environment Variables

```

# .env (in project root, never commit to git):
VITE_API_URL=http://localhost:3000

# .env.production:
VITE_API_URL=https://api.your-production-domain.com

# .gitignore — always add:
.env

```

```
.env.*
!.env.example

# Access in code:
const BASE_URL = import.meta.env.VITE_API_URL;
// Note: All Vite env vars MUST start with VITE_ to be accessible in the browser.
```

2.6 VSCode Extensions (Mandatory)

Extension	Purpose
ES7+ React/Redux/React-Native snippets	Fast component and hook scaffolding
Tailwind CSS IntelliSense	Autocomplete for Tailwind classes
Prettier - Code formatter	Consistent code formatting on save
ESLint	Catches code quality issues in real time
TypeScript Hero	Auto-imports and TypeScript helpers
Auto Rename Tag	Renames opening and closing JSX tags together
GitLens	See who wrote each line, when, and why

3. Folder Structure — Strict & ERP-Aligned

The folder structure mirrors the NestJS backend module-by-module. This is intentional — when a backend engineer says "the inventory module", the frontend engineer knows exactly which folder to open.

```

src/
├── main.tsx                ← Application entry point
├── App.tsx                 ← Root component + router setup
├── modules/                ← One folder per backend module
│   ├── inventory/
│   │   ├── pages/
│   │   │   ├── ProductListPage.tsx
│   │   │   └── ProductDetailPage.tsx
│   │   ├── components/
│   │   │   ├── ProductTable.tsx
│   │   │   ├── ProductForm.tsx
│   │   │   └── StockBadge.tsx
│   │   ├── hooks/
│   │   │   └── useProducts.ts
│   │   └── services/
│   │       └── productService.ts
│   ├── sales/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── hooks/
│   │   └── services/
│   ├── crm/
│   ├── finance/
│   └── auth/
│       ├── pages/
│       │   └── LoginPage.tsx
│       ├── components/
│       └── services/
│           └── authService.ts
├── shared/
│   ├── components/        ← Reusable across ALL modules
│   │   ├── DataTable/
│   │   │   ├── DataTable.tsx
│   │   │   └── DataTablePagination.tsx
│   │   ├── Modal/
│   │   ├── StatusBadge/
│   │   └── PageHeader/
│   ├── ui/                ← shadcn/ui base components
│   │   ├── button.tsx
│   │   ├── input.tsx
│   │   ├── dialog.tsx
│   │   └── toast.tsx
│   ├── utils/
│   │   └── formatters.ts   ← Currency, date, number formatters

```


├──	└── validators.ts	
├──	└── constants/	
	└── routes.ts	← All route paths in one place
	└── queryKeys.ts	← All TanStack Query keys in one place
├──	hooks/	← Global custom hooks
	└── useAuth.ts	
	└── useDebounce.ts	
	└── usePagination.ts	
├──	lib/	← Library configuration
	└── axios.ts	← Configured Axios instance
	└── queryClient.ts	← TanStack Query client config
	└── store.ts	← Zustand global store
├──	routes/	
	└── AppRouter.tsx	← All route definitions
	└── ProtectedRoute.tsx	← JWT-guarded route wrapper
├──	types/	
└──	└── api.types.ts	← API response/request shapes (mirrors backend DTOs)
	└── inventory.types.ts	
	└── sales.types.ts	
	└── auth.types.ts	

3.1 Folder Structure Rules

RULE 1: Never put module-specific logic in shared/. If it is only used by inventory, it lives in modules/inventory/.

RULE 2: Folder names must exactly match backend module names. If the backend has an "inventory" module, the folder is modules/inventory/ — not modules/products/ or modules/stock/.

RULE 3: Pages are full-screen views. Components are building blocks. Hooks extract logic. Services make API calls. Never mix these responsibilities.

4. TypeScript in ThunderERP — The Type System

TypeScript is not optional in ThunderERP. Every piece of data that crosses the API boundary must have a TypeScript type. This is the primary tool that catches frontend-backend contract mismatches before they reach production.

4.1 Mirroring Backend DTOs

Every NestJS DTO has a corresponding TypeScript interface in the frontend `types/` folder. These must be kept in sync. The Swagger page at <http://localhost:3000/api> is the source of truth.

```
// src/types/inventory.types.ts

// Mirror of NestJS ProductDto (backend source of truth):
export interface ProductDto {
  id: number;
  name: string;
  price: number;
  gstPercentage: number;
  quantity: number;
  category: string | null;
  unit: string;
  isActive: boolean;
  createdAt: string; // ISO date string from JSON
  updatedAt: string;
}

// Mirror of NestJS CreateProductDto:
export interface CreateProductDto {
  name: string;
  price: number;
  gstPercentage: number;
  quantity: number;
  category?: string;
  unit?: string;
}

// Paginated response wrapper (mirrors backend PaginationResult):
export interface PaginatedResponse<T> {
  data: T[];
  meta: {
    total: number;
    page: number;
    limit: number;
    totalPages: number;
    hasNextPage: boolean;
  };
}

// Standard API response wrapper:
```

```
export interface ApiResponse<T> {
  success: boolean;
  data: T;
  timestamp: string;
}
```

4.2 Common TypeScript Patterns in React

```
// Typing component props:
interface ProductCardProps {
  product: ProductDto;
  onEdit: (id: number) => void;
  onDelete: (id: number) => void;
  isLoading?: boolean;
}

function ProductCard({ product, onEdit, onDelete, isLoading = false }:
ProductCardProps) {
  return <div>...</div>;
}

// Typing useState:
const [selectedProduct, setSelectedProduct] = useState<ProductDto | null>(null);
const [products, setProducts] = useState<ProductDto[]>([]);

// Typing event handlers:
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  setValue(e.target.value);
};

// Typing async functions:
const fetchProduct = async (id: number): Promise<ProductDto> => {
  const response = await api.get<ApiResponse<ProductDto>>(`/products/${id}`);
  return response.data.data;
};

// Union types for status:
type OrderStatus = 'DRAFT' | 'PENDING' | 'CONFIRMED' | 'SHIPPED' | 'DELIVERED' |
'COMPLETED' | 'CANCELLED';
```

4.3 TypeScript Rules

Rule	Why & What to Do Instead
Never use "any"	"any" disables type checking entirely. Use "unknown" and narrow the type, or define a proper interface.
Never use type assertions blindly	value as ProductDto can hide bugs. Validate the shape first.

Always type API responses	<code>api.get<ApiResponse<ProductDto[]>>()</code> — never <code>api.get()</code> without a type.
Use strict mode in tsconfig	Catches null/undefined errors that would crash in production.
Use enums or union types for status	Prevents typos in status comparisons. "PENDINGG" vs "PENDING".

5. React Core Concepts — ERP Context

5.1 Functional Components

ThunderERP uses functional components exclusively. No class components. Every component is a JavaScript function that accepts props and returns JSX.

```
// Basic functional component:
function ProductCard({ product }: { product: ProductDto }) {
  return (
    <div className="border rounded-lg p-4">
      <h3 className="font-semibold">{product.name}</h3>
      <p className="text-gray-600">₹{product.price.toFixed(2)}</p>
    </div>
  );
}

// Arrow function style (same thing, different syntax):
const ProductCard = ({ product }: { product: ProductDto }) => {
  return <div>...</div>;
};

// Export (always at the bottom, or as default):
export default ProductCard;
// OR: export { ProductCard };
```

5.2 The Core Hooks

useState — Local UI State

```
// Basic state:
const [isOpen, setIsOpen] = useState(false);
const [searchQuery, setSearchQuery] = useState("");

// Object state (form fields):
const [filters, setFilters] = useState({
  status: "",
  category: "",
  page: 1,
});

// Update one field without touching others:
setFilters(prev => ({ ...prev, status: "PENDING" }));

// Rule: useState is for UI state (modal open, selected rows, search input).
// Use TanStack Query for data from the backend — not useState.
```

useEffect — Side Effects

```
// Run once on mount:
useEffect(() => {
  document.title = "Products | ThunderERP";
}, []); // Empty array = run once

// Run when dependency changes:
useEffect(() => {
  // Fetch is handled by TanStack Query — this is just a setup example
  console.log("Search changed:", searchQuery);
}, [searchQuery]);

// Cleanup (runs before next effect or on unmount):
useEffect(() => {
  const timer = setTimeout(() => setDebounceQuery(searchQuery), 300);
  return () => clearTimeout(timer); // Cleanup
}, [searchQuery]);

// RULE: Avoid fetching data in useEffect — use TanStack Query instead.
// useEffect data fetching has no caching, no loading state, no deduplication.
```

useMemo & useCallback — Performance

```
// useMemo — memoize an expensive computed value:
const filteredProducts = useMemo(() => {
  return products.filter(p => p.name.toLowerCase().includes(search.toLowerCase()));
}, [products, search]); // Only recomputes when products or search changes

// useCallback — memoize a function (prevent child re-renders):
const handleDelete = useCallback(async (id: number) => {
  await productService.softDelete(id);
  queryClient.invalidateQueries({ queryKey: QUERY_KEYS.products });
}, [queryClient]);

// RULE: Do NOT use useMemo/useCallback everywhere.
// Only use when: (1) expensive calculation, or (2) passing to memoized child.
// Premature optimization adds complexity without benefit.
```

useRef — DOM References & Persistent Values

```
// Reference a DOM element:
const inputRef = useRef<HTMLInputElement>(null);

useEffect(() => {
  inputRef.current?.focus(); // Focus input on mount
}, []);

return <input ref={inputRef} />;

// Store a value that does NOT trigger re-render:
const previousValueRef = useRef(value);
useEffect(() => {
```

```
previousValueRef.current = value;  
});
```

5.3 Custom Hooks — Extract and Reuse Logic

Custom hooks are the primary way to share logic between components. If two components do the same thing, that logic becomes a custom hook.

```
// src/hooks/useDebounce.ts  
// Delays updating a value until the user stops typing  
export function useDebounce<T>(value: T, delay: number = 300): T {  
  const [debouncedValue, setDebouncedValue] = useState(value);  
  
  useEffect(() => {  
    const timer = setTimeout(() => setDebouncedValue(value), delay);  
    return () => clearTimeout(timer);  
  }, [value, delay]);  
  
  return debouncedValue;  
}  
  
// Usage — search with debounce:  
const [searchInput, setSearchInput] = useState("");  
const debouncedSearch = useDebounce(searchInput, 400);  
  
// TanStack Query will only fire when debouncedSearch changes (after 400ms of no  
typing):  
const { data } = useQuery({  
  queryKey: ["products", { search: debouncedSearch }],  
  queryFn: () => productService.getAll({ search: debouncedSearch }),  
});
```

6. API Integration — The Bridge to NestJS

This is the most critical section for daily development. Every interaction with the backend follows this exact pattern. Deviating from it creates unmaintainable code.

6.1 Axios Instance — Centralized Configuration

Never import Axios directly in a component or service. Always use the configured instance from `lib/axios.ts`. This ensures every request has the JWT token, base URL, and error handling applied automatically.

```
// src/lib/axios.ts
import axios from "axios";
import { useAuthStore } from "../store";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  timeout: 15000,
  headers: { "Content-Type": "application/json" },
});

// Request interceptor - attach JWT token to every request:
api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().accessToken;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Response interceptor - handle 401 globally:
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401) {
      useAuthStore.getState().logout();
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);

export default api;
```

6.2 Service Layer — Never Call API in Components

All API calls live in service files. Components never call Axios directly. This separation means you can change the API URL or request format in one place without touching any component.

```
// src/modules/inventory/services/productService.ts
```



```
import api from "@lib/axios";
import type { ProductDto, CreateProductDto, PaginatedResponse } from
"@types/inventory.types";

export const productService = {

  getAll: async (params?: { page?: number; limit?: number; search?: string }) => {
    const response = await api.get<{ data: PaginatedResponse<ProductDto>
}>("/products", { params });
    return response.data.data;
  },

  getById: async (id: number) => {
    const response = await api.get<{ data: ProductDto }>(`/products/${id}`);
    return response.data.data;
  },

  create: async (dto: CreateProductDto) => {
    const response = await api.post<{ data: ProductDto }>("/products", dto);
    return response.data.data;
  },

  update: async (id: number, dto: Partial<CreateProductDto>) => {
    const response = await api.patch<{ data: ProductDto }>(`/products/${id}`, dto);
    return response.data.data;
  },

  softDelete: async (id: number) => {
    const response = await api.delete(`/products/${id}`);
    return response.data;
  },
};
```

6.3 TanStack Query — Server State Management

TanStack Query handles all data that comes from the backend. It provides automatic caching, background refetching, loading states, error states, and cache invalidation. Do not use `useState` + `useEffect` for fetching data.

Query Keys — The Cache Identity

```
// src/shared/constants/queryKeys.ts
// Centralize ALL query keys to avoid typos and enable precise cache control
export const QUERY_KEYS = {
  products: {
    all: ["products"] as const,
    lists: () => [...QUERY_KEYS.products.all, "list"] as const,
    list: (filters: object) => [...QUERY_KEYS.products.lists(), filters] as const,
    detail: (id: number) => [...QUERY_KEYS.products.all, "detail", id] as const,
  },
  orders: {
```

```

    all: ["orders"] as const,
    list: (filters: object) => ["orders", "list", filters] as const,
    detail: (id: number) => ["orders", "detail", id] as const,
  },
  customers: {
    all: ["customers"] as const,
    detail: (id: number) => ["customers", "detail", id] as const,
  },
};

```

useQuery — Fetching Data

```

// src/modules/inventory/hooks/useProducts.ts
import { useQuery } from "@tanstack/react-query";
import { productService } from "../services/productService";
import { QUERY_KEYS } from "@shared/constants/queryKeys";

export function useProducts(filters = {}) {
  return useQuery({
    queryKey: QUERY_KEYS.products.list(filters),
    queryFn: () => productService.getAll(filters),
    staleTime: 1000 * 60 * 5, // Data is fresh for 5 minutes – no refetch
    retry: 2, // Retry failed requests twice
  });
}

// In component:
function ProductListPage() {
  const [filters, setFilters] = useState({ page: 1, limit: 20 });
  const { data, isLoading, isError, error } = useProducts(filters);

  if (isLoading) return <TableSkeleton />;
  if (isError) return <ErrorState message={error.message} />;

  return <ProductTable data={data?.data ?? []} meta={data?.meta} />;
}

```

useMutation — Creating, Updating, Deleting

```

// src/modules/inventory/hooks/useProductMutations.ts
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { toast } from "sonner";

export function useCreateProduct() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: productService.create,

    onSuccess: (newProduct) => {
      // Invalidate the products list so it refetches automatically:
    }
  });
}

```

```

        queryClient.invalidateQueries({ queryKey: QUERY_KEYS.products.lists() });
        toast.success(`Product "${newProduct.name}" created successfully`);
    },

    onError: (error: AxiosError<ApiErrorResponse>) => {
        const message = error.response?.data?.message ?? "Failed to create product";
        toast.error(message);
    },
});
}

// In component:
const { mutate: createProduct, isPending } = useCreateProduct();

const handleSubmit = (data: CreateProductDto) => {
    createProduct(data);
    // TanStack Query handles loading, success, and error automatically.
};

```

6.4 Query Client Configuration

```

// src/lib/queryClient.ts
import { QueryClient } from "@tanstack/react-query";

export const queryClient = new QueryClient({
    defaultOptions: {
        queries: {
            staleTime: 1000 * 60 * 2, // 2 minutes before data is considered stale
            retry: (failureCount, error) => {
                // Do not retry on 4xx errors (client mistakes):
                if (error?.response?.status >= 400 && error?.response?.status < 500) return
false;
                return failureCount < 2;
            },
        },
        mutations: {
            retry: false, // Never retry mutations automatically
        },
    },
});

// src/main.tsx - wrap the app:
import { QueryClientProvider } from "@tanstack/react-query";
import { ReactQueryDevtools } from "@tanstack/react-query-devtools";

ReactDOM.createRoot(document.getElementById("root")!).render(
    <QueryClientProvider client={queryClient}>
        <App />
        <ReactQueryDevtools initialIsOpen={false} /> {/* Dev tool - remove in prod */}
    </QueryClientProvider>
);

```

7. Routing System

ThunderERP uses React Router v6. Every route maps 1:1 to a backend module. The routing system also handles authentication — unauthenticated users are redirected to /login automatically.

7.1 Route Map

Route	Page & Description
/login	LoginPage — Public. JWT token obtained here.
/dashboard	DashboardPage — Protected. KPI cards + charts.
/inventory/products	ProductListPage — Protected. Table with search, filter, pagination.
/inventory/products/:id	ProductDetailPage — Protected. Single product view + edit.
/sales/orders	OrderListPage — Protected. All orders with status filters.
/sales/orders/:id	OrderDetailPage — Protected. Order details + status workflow.
/crm/leads	LeadListPage — Protected. CRM leads pipeline.
/crm/customers/:id	CustomerDetailPage — Protected. Customer 360 view.
/finance/invoices	InvoiceListPage — Protected. Invoice management.
/finance/invoices/:id	InvoiceDetailPage — Protected. Invoice + payment recording.

7.2 Router Implementation

```
// src/routes/AppRouter.tsx
import { createBrowserRouter, RouterProvider, Outlet } from "react-router-dom";
import { ProtectedRoute } from "../ProtectedRoute";
import { AppLayout } from "@shared/components/AppLayout";

const router = createBrowserRouter([
  {
    path: "/login",
    element: <LoginPage />,
  },
  {
    element: <ProtectedRoute />, // Guards all children
    children: [
      {
        element: <AppLayout />, // Sidebar + header wrapper
        children: [
          { path: "/dashboard", element: <DashboardPage /> },
          { path: "/inventory/products", element: <ProductListPage /> },
          { path: "/inventory/products/:id", element: <ProductDetailPage /> },
          { path: "/sales/orders", element: <OrderListPage /> },
          { path: "/sales/orders/:id", element: <OrderDetailPage /> },
          { path: "/crm/leads", element: <LeadListPage /> },
          { path: "/finance/invoices", element: <InvoiceListPage /> },
        ]
      }
    ]
  }
])
```

```

        ],
      }
    ]
  }
});

export function AppRouter() {
  return <RouterProvider router={router} />;
}

```

7.3 ProtectedRoute — JWT Guard

```

// src/routes/ProtectedRoute.tsx
import { Navigate, Outlet } from "react-router-dom";
import { useAuthStore } from "@lib/store";

export function ProtectedRoute() {
  const accessToken = useAuthStore(state => state.accessToken);

  if (!accessToken) {
    return <Navigate to="/login" replace />;
  }

  return <Outlet />; // Render the child route
}

```

7.4 Navigation

```

// Programmatic navigation:
import { useNavigate } from "react-router-dom";
const navigate = useNavigate();

navigate('/inventory/products');
navigate(`/sales/orders/${order.id}`);
navigate(-1); // Go back

// Link component (preferred for clickable elements):
import { Link } from "react-router-dom";
<Link to="/inventory/products">View Products</Link>

// Route params:
import { useParams } from "react-router-dom";
const { id } = useParams<{ id: string }>();
const productId = parseInt(id!, 10);

// Query string:
import { useSearchParams } from "react-router-dom";
const [searchParams, setSearchParams] = useSearchParams();
const page = Number(searchParams.get('page') ?? '1');

```

8. Authentication Flow

Authentication is one of the most critical parts of the frontend. A mistake here means unauthorized access to ERP data. Follow this pattern exactly.

8.1 Auth Flow Overview

```
1. User submits login form
2. POST /auth/login { email, password }
3. Backend returns { access_token, user: { id, email, role, name } }
4. Frontend stores access_token in Zustand (memory – NOT localStorage)
5. Axios interceptor attaches token to every subsequent request
6. ProtectedRoute checks for token – renders page if present
7. On 401 response → Axios interceptor clears store → redirect to /login
8. On page refresh → token is lost (memory cleared) → user re-authenticates

// Note on security: localStorage is vulnerable to XSS attacks.
// Storing tokens in memory (Zustand) is safer for ERP applications.
// For longer sessions, implement a refresh token in HttpOnly cookies via backend.
```

8.2 Auth Store — Zustand

```
// src/lib/store.ts
import { create } from "zustand";
import { persist } from "zustand/middleware";

interface User {
  id: number;
  email: string;
  name: string;
  role: string;
}

interface AuthStore {
  accessToken: string | null;
  user: User | null;
  setAuth: (token: string, user: User) => void;
  logout: () => void;
}

export const useAuthStore = create<AuthStore>(() => ({
  set: () => ({
    accessToken: null,
    user: null,
    setAuth: (token, user) => set({ accessToken: token, user }),
    logout: () => set({ accessToken: null, user: null }),
  })
}));
```

8.3 Login Page Implementation

```
// src/modules/auth/pages/LoginPage.tsx
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

const loginSchema = z.object({
  email: z.string().email("Enter a valid email"),
  password: z.string().min(6, "Password must be at least 6 characters"),
});

type LoginFormData = z.infer<typeof loginSchema>;

export function LoginPage() {
  const navigate = useNavigate();
  const { setAuth } = useAuthStore();

  const { register, handleSubmit, formState: { errors } } =
    useForm<LoginFormData>({
      resolver: zodResolver(loginSchema),
    });

  const loginMutation = useMutation({
    mutationFn: (data: LoginFormData) => authService.login(data),
    onSuccess: ({ access_token, user }) => {
      setAuth(access_token, user);
      navigate("/dashboard");
    },
    onError: (error) => {
      toast.error("Invalid email or password");
    }
  });

  return (
    <form onSubmit={handleSubmit(data => loginMutation.mutate(data))}
      className="max-w-sm mx-auto mt-20 space-y-4">
      <h1 className="text-2xl font-bold text-primary">ThunderERP Login</h1>
      <input {...register("email")} placeholder="Email" className="input" />
      {errors.email && <p className="text-red-500">{errors.email.message}</p>}
      <input {...register("password")} type="password" placeholder="Password" />
      {errors.password && <p
className="text-red-500">{errors.password.message}</p>}
      <button type="submit" disabled={loginMutation.isPending}>
        {loginMutation.isPending ? "Logging in..." : "Login"}
      </button>
    </form>
  );
}
```

8.4 Role-Based UI Rendering

```
// Show/hide UI elements based on user role:
const { user } = useAuthStore();

// Conditional rendering:
{(user?.role === "admin" || user?.role === "manager") && (
  <button onClick={handleDelete}>Delete Product</button>
)}

// Custom hook for cleaner role checks:
export function usePermissions() {
  const { user } = useAuthStore();
  return {
    canDelete: user?.role === "admin",
    canEdit: ["admin", "manager"].includes(user?.role ?? ""),
    canViewFinance: ["admin", "finance"].includes(user?.role ?? ""),
  };
}

// Usage:
const { canDelete, canEdit } = usePermissions();
{canDelete && <DeleteButton />}

// CRITICAL: This is UI-only protection.
// The backend ALWAYS enforces actual permissions via guards.
// Never rely on frontend role checks alone for security.
```


9. Forms — React Hook Form + Zod

All forms in ThunderERP use React Hook Form for performance and Zod for validation. Zod schemas mirror the NestJS DTO validation rules exactly — if the backend requires a minimum length of 2 characters on a name, the Zod schema must enforce the same.

9.1 Why React Hook Form?

Problem with plain controlled forms	React Hook Form solution
Every keystroke re-renders the whole form	Uncontrolled inputs — re-renders only on validation/submit
Manual validation logic everywhere	Schema-based validation with Zod resolver
Complex nested form state management	useFieldArray for dynamic fields
Unclear dirty/touched/valid states	Automatic formState tracking

9.2 Complete Form Example — Create Product

```
// src/modules/inventory/components/ProductForm.tsx
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

// Zod schema mirrors NestJS CreateProductDto validation rules:
const productSchema = z.object({
  name: z.string().min(2, "Name must be at least 2 characters").max(100),
  price: z.number({ invalid_type_error: "Price must be a number" })
    .positive("Price must be positive"),
  gstPercentage: z.number().min(0).max(100),
  quantity: z.number().int("Quantity must be a whole number").min(0),
  category: z.string().optional(),
  unit: z.string().default("pcs"),
});

type ProductFormData = z.infer<typeof productSchema>;

interface ProductFormProps {
  onSubmit: (data: ProductFormData) => void;
  defaultValues?: Partial<ProductFormData>;
  isLoading?: boolean;
}

export function ProductForm({ onSubmit, defaultValues, isLoading }: ProductFormProps) {
```

```

    const { register, handleSubmit, formState: { errors, isDirty } } =
useForm<ProductFormData>({
  resolver: zodResolver(productSchema),
  defaultValues: defaultValues ?? { unit: "pcs", gstPercentage: 18 },
});

return (
  <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">

    <div>
      <label className="label">Product Name *</label>
      <input {...register("name")} className="input" placeholder="e.g. Widget
Pro" />
      {errors.name && <p className="error">{errors.name.message}</p>}
    </div>

    <div className="grid grid-cols-2 gap-4">
      <div>
        <label className="label">Price (₹) *</label>
        <input {...register("price", { valueAsNumber: true })} type="number"
          step="0.01" className="input" />
        {errors.price && <p className="error">{errors.price.message}</p>}
      </div>
      <div>
        <label className="label">GST % *</label>
        <input {...register("gstPercentage", { valueAsNumber: true })}
type="number"
          step="0.01" className="input" />
      </div>
    </div>

    <button type="submit" disabled={isLoading || !isDirty}
className="btn-primary">
      {isLoading ? "Saving..." : "Save Product"}
    </button>

  </form>
);
}

```

9.3 Dynamic Form Fields with useFieldArray

Order items are a dynamic list — users add and remove line items. `useFieldArray` manages this automatically.

```

// Order creation form with dynamic line items:
const { fields, append, remove } = useFieldArray({
  control,          // from useForm
  name: "items",    // field array name in schema
});

// Render each item:
{fields.map((field, index) => (

```

```
<div key={field.id} className="flex gap-2 items-end">
  <select {...register(`items.${index}.productId`, { valueAsNumber: true })}>
    {products.map(p => <option key={p.id} value={p.id}>{p.name}</option>)}
  </select>
  <input {...register(`items.${index}.quantity`, { valueAsNumber: true })}
type="number" />
  <button type="button" onClick={() => remove(index)}>Remove</button>
</div>
))}

<button type="button"
  onClick={() => append({ productId: 0, quantity: 1 })}
  className="btn-secondary">
  + Add Item
</button>
```

10. ERP UI Patterns — The Standard Components

These are the non-negotiable UI patterns used across all modules. Building them correctly once in `shared/components/` means every module gets them for free.

10.1 Data Tables

Every list view in ThunderERP uses the same `DataTable` component. It supports sorting, filtering, pagination, row selection, and bulk actions out of the box.

```
// src/shared/components/DataTable/DataTable.tsx
// Built on TanStack Table (formerly React Table)
npm install @tanstack/react-table

import { useReactTable, getCoreRowModel, flexRender } from "@tanstack/react-table";

interface DataTableProps<T> {
  data: T[];
  columns: ColumnDef<T>[];
  isLoading?: boolean;
  meta?: PaginationMeta;
  onPageChange?: (page: number) => void;
}

export function DataTable<T>({ data, columns, isLoading, meta, onPageChange }:
  DataTableProps<T>) {
  const table = useReactTable({
    data,
    columns,
    getCoreRowModel: getCoreRowModel(),
    manualPagination: true, // Backend handles pagination
    pageCount: meta?.totalPages ?? 0,
  });

  if (isLoading) return <TableSkeleton columns={columns.length} />;

  return (
    <div>
      <table className="w-full border-collapse">
        <thead className="bg-primary text-white">
          {table.getHeaderGroups().map(hg => (
            <tr key={hg.id}>
              {hg.headers.map(h => (
                <th key={h.id} className="p-3 text-left text-sm font-semibold">
                  {flexRender(h.column.columnDef.header, h.getContext())}
                </th>
              ))}
            </tr>
          ))}
        </thead>
        <tbody>
```

```

        {table.getRowModel().rows.map((row, i) => (
          <tr key={row.id} className={i % 2 === 0 ? "bg-white" : "bg-blue-50"}>
            {row.getVisibleCells().map(cell => (
              <td key={cell.id} className="p-3 text-sm border-b">
                {flexRender(cell.column.columnDef.cell, cell.getContext())}
              </td>
            ))}
          </tr>
        ))}
      </tbody>
    </table>
    {meta && <DataTablePagination meta={meta} onPageChange={onPageChange} />}
  </div>
);
}

```

Defining Columns

```

// src/modules/inventory/components/ProductTable.tsx
import { ColumnDef } from "@tanstack/react-table";

export const productColumns: ColumnDef<ProductDto>[] = [
  {
    accessorKey: "name",
    header: "Product Name",
    cell: ({ row }) => <span className="font-medium">{row.original.name}</span>,
  },
  {
    accessorKey: "price",
    header: "Price",
    cell: ({ row }) => <span>{formatCurrency(row.original.price)}</span>,
  },
  {
    accessorKey: "quantity",
    header: "Stock",
    cell: ({ row }) => <StockBadge quantity={row.original.quantity} />,
  },
  {
    id: "actions",
    header: "Actions",
    cell: ({ row }) => <ProductRowActions product={row.original} />,
  },
];

```

10.2 Status Badges

Every status field (OrderStatus, InvoiceStatus, LeadStatus) renders as a colored badge. Define ALL badge variants in one place.

```

// src/shared/components/StatusBadge/StatusBadge.tsx
const STATUS_STYLES: Record<string, string> = {
  DRAFT: "bg-gray-100 text-gray-700",

```

```

    PENDING:      "bg-yellow-100 text-yellow-700",
    CONFIRMED:     "bg-blue-100 text-blue-700",
    SHIPPED:       "bg-purple-100 text-purple-700",
    DELIVERED:     "bg-teal-100 text-teal-700",
    COMPLETED:    "bg-green-100 text-green-700",
    CANCELLED:     "bg-red-100 text-red-700",
    UNPAID:        "bg-red-100 text-red-700",
    PARTIALLY_PAID: "bg-orange-100 text-orange-700",
    PAID:          "bg-green-100 text-green-700",
  };

export function StatusBadge({ status }: { status: string }) {
  const style = STATUS_STYLES[status] ?? "bg-gray-100 text-gray-600";
  return (
    <span className={`inline-flex px-2 py-1 rounded-full text-xs font-medium
    ${style}`}>
      {status.replace(/_/g, " ")}
    </span>
  );
}

```

10.3 Modals — Create & Edit

Secondary actions (create, edit, confirm delete) use modals. Full-page navigation is reserved for primary views only.

```

// src/modules/inventory/pages/ProductListPage.tsx
export function ProductListPage() {
  const [isCreateOpen, setIsCreateOpen] = useState(false);
  const [editingProduct, setEditingProduct] = useState<ProductDto | null>(null);
  const { mutate: createProduct, isPending } = useCreateProduct();

  return (
    <div>
      <PageHeader title="Products">
        <button onClick={() => setIsCreateOpen(true)} className="btn-primary">
          + Add Product
        </button>
      </PageHeader>

      <ProductTable onEdit={setEditingProduct} />

      {/* Create Modal */}
      <Dialog open={isCreateOpen} onOpenChange={setIsCreateOpen}>
        <DialogContent>
          <DialogTitle>New Product</DialogTitle>
          <ProductForm
            onSubmit={(data) => {
              createProduct(data, { onSuccess: () => setIsCreateOpen(false) });
            }}
            isLoading={isPending}
          />
        </DialogContent>
      </Dialog>
    </div>
  );
}

```

```

        </DialogContent>
      </Dialog>

      { /* Edit Modal */ }
      <Dialog open={!editingProduct} onOpenChange={() => setEditingProduct(null)}>
        <DialogContent>
          <DialogTitle>Edit Product</DialogTitle>
          {editingProduct && <ProductForm defaultValues={editingProduct} ... />}
        </DialogContent>
      </Dialog>
    </div>
  );
}

```

10.4 Loading Skeletons

Never show a blank screen while data loads. Use skeleton placeholders that match the shape of the content.

```

// src/shared/components/TableSkeleton.tsx
export function TableSkeleton({ rows = 5, columns = 4 }) {
  return (
    <div className="space-y-2">
      {Array.from({ length: rows }).map((_, i) => (
        <div key={i} className="flex gap-4">
          {Array.from({ length: columns }).map((_, j) => (
            <div key={j}
              className="h-10 bg-gray-200 rounded animate-pulse flex-1" />
          ))}
        </div>
      ))}
    </div>
  );
}

// KPI Card Skeleton:
export function KpiSkeleton() {
  return (
    <div className="border rounded-lg p-4 space-y-2">
      <div className="h-4 bg-gray-200 rounded animate-pulse w-24" />
      <div className="h-8 bg-gray-200 rounded animate-pulse w-32" />
    </div>
  );
}

```

10.5 Currency & Date Formatters

```

// src/shared/utils/formatters.ts

// Indian Rupee currency format:
export function formatCurrency(amount: number | string): string {

```

```
    return new Intl.NumberFormat("en-IN", {
      style: "currency",
      currency: "INR",
      minimumFractionDigits: 2,
    }).format(Number(amount));
  }
  // Output: ₹1,23,456.78

  // Date formatting:
  export function formatDate(date: string | Date): string {
    return new Intl.DateTimeFormat("en-IN", {
      day: "2-digit", month: "short", year: "numeric"
    }).format(new Date(date));
  }
  // Output: 15 Jan 2025

  // Relative time (e.g., "3 days ago"):
  export function timeAgo(date: string | Date): string {
    const seconds = Math.floor((Date.now() - new Date(date).getTime()) / 1000);
    if (seconds < 60) return "just now";
    if (seconds < 3600) return `${Math.floor(seconds/60)} min ago`;
    if (seconds < 86400) return `${Math.floor(seconds/3600)} hr ago`;
    return `${Math.floor(seconds/86400)} days ago`;
  }
}
```


11. State Management

ThunderERP uses two tools for state. The choice between them is straightforward: server data goes to TanStack Query, everything else goes to Zustand.

State Type	Tool	Examples
Server / Remote State	TanStack Query	Products list, order details, customer records — anything from the API.
Global App State	Zustand	Auth token, current user, theme preference, notification queue.
Local UI State	useState	Modal open/close, form field values, selected row, accordion open.
Derived / Computed State	useMemo	Filtered list, calculated totals, formatted data — derived from other state.

 **KEY RULE:** Never duplicate server data in Zustand. If you have products in TanStack Query cache, do not copy them to a Zustand store. TanStack Query is already the single source of truth for server data.

11.1 Zustand — Global App Store

```
// src/lib/store.ts
import { create } from "zustand";

// Auth slice:
interface AuthSlice {
  accessToken: string | null;
  user: User | null;
  setAuth: (token: string, user: User) => void;
  logout: () => void;
}

// UI slice:
interface UISlice {
  sidebarOpen: boolean;
  setSidebarOpen: (open: boolean) => void;
  theme: "light" | "dark";
  setTheme: (theme: "light" | "dark") => void;
}

export const useAuthStore = create<AuthSlice>() (set => ({
  accessToken: null,
  user: null,
  setAuth: (token, user) => set({ accessToken: token, user }),
  logout: () => set({ accessToken: null, user: null }),
}));
```

```
export const useUIStore = create<UISlice>() (set => ({
  sidebarOpen: true,
  setSidebarOpen: open => set({ sidebarOpen: open }),
  theme: "light",
  setTheme: theme => set({ theme }),
}));

// Usage (read only what you need to avoid unnecessary re-renders):
const user = useAuthStore(state => state.user);
const sidebarOpen = useUIStore(state => state.sidebarOpen);
```

11.2 TanStack Query Cache Invalidation

When a mutation succeeds (e.g., a product is created), you must tell TanStack Query to refetch the product list. This is done by invalidating the relevant query keys.

```
const queryClient = useQueryClient();

// After creating a product - invalidate the list:
queryClient.invalidateQueries({ queryKey: QUERY_KEYS.products.lists() });

// After updating a specific product - invalidate the detail AND the list:
queryClient.invalidateQueries({ queryKey: QUERY_KEYS.products.detail(productId) });
queryClient.invalidateQueries({ queryKey: QUERY_KEYS.products.lists() });

// Optimistic update (update the UI before the server responds):
onMutate: async (updatedProduct) => {
  // Cancel in-flight queries to prevent overwrite:
  await queryClient.cancelQueries({ queryKey: QUERY_KEYS.products.lists() });

  // Snapshot current data (for rollback on error):
  const previous = queryClient.getQueryData(QUERY_KEYS.products.lists());

  // Apply optimistic update immediately:
  queryClient.setQueryData(QUERY_KEYS.products.lists(), (old) =>
    old?.map(p => p.id === updatedProduct.id ? { ...p, ...updatedProduct } : p)
  );

  return { previous }; // Context for rollback
},

onError: (err, vars, context) => {
  // Rollback on error:
  queryClient.setQueryData(QUERY_KEYS.products.lists(), context?.previous);
},
```

12. Error Handling

In a financial ERP, silent failures are unacceptable. The user must always know whether an action succeeded, failed, or is still in progress. Every API call has three states: loading, success, and error — and all three must be handled.

12.1 Error Handling Layers

Layer	What It Handles
Axios Response Interceptor	Global 401 (auto-logout), 500 (generic error toast), network errors.
TanStack Query onError	Per-query error handling. Shows specific error messages from backend.
Form Validation (Zod)	Client-side validation before request is even made. Immediate user feedback.
React Error Boundary	Catches unexpected JavaScript errors in components. Shows fallback UI.
Empty States	When API returns empty array — shows illustration + helpful message.

12.2 Toast Notifications — Sonner

```
// src/main.tsx - add Toaster to app root:
import { Toaster } from "sonner";
<Toaster position="top-right" richColors />

// Usage in mutations:
import { toast } from "sonner";

toast.success("Order confirmed successfully");
toast.error("Failed to update stock. Please try again.");
toast.warning("Low stock: Widget Pro has only 2 units remaining.");
toast.info("Invoice #INV-2024-001 sent to customer.");

// With action button:
toast.success("Product deleted", {
  action: {
    label: "Undo",
    onClick: () => undoDelete(productId),
  },
},
);
```

12.3 Error Boundary

```
// src/shared/components/ErrorMessage.tsx
import { Component, ReactNode } from "react";
```

```
interface Props { children: ReactNode; fallback?: ReactNode; }
interface State { hasError: boolean; }

export class ErrorBoundary extends Component<Props, State> {
  state = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error: Error, info: { componentStack: string }) {
    console.error("Component crashed:", error, info);
    // In production, log to error tracking service (Sentry, etc.)
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback ?? (
        <div className="text-center py-10">
          <p className="text-red-500 font-medium">Something went wrong.</p>
          <button onClick={() => this.setState({ hasError: false })}
            className="btn-secondary mt-4">Try Again</button>
        </div>
      );
    }
    return this.props.children;
  }
}

// Wrap each page/module:
<ErrorBoundary><ProductListPage /></ErrorBoundary>
```

13. Building a Complete Feature — Step by Step

This walkthrough shows how to build the Inventory Products module from scratch. Use this exact workflow for every new feature you build.

The 8-Step Feature Workflow

5. Understand the backend — open Swagger at <http://localhost:3000/api> and read the endpoints.
6. Define TypeScript types — create `types/inventory.types.ts` mirroring the backend DTOs.
7. Create the service — `modules/inventory/services/productService.ts`.
8. Create custom hooks — `modules/inventory/hooks/useProducts.ts`.
9. Build components bottom-up — start with the simplest (ProductCard), build up to the page.
10. Create the page — `modules/inventory/pages/ProductListPage.tsx`.
11. Add the route — register in `routes/AppRouter.tsx`.
12. Test in browser with Postman running in parallel to verify API responses.

Step-by-Step Code Walk

Step 1 — Types (`types/inventory.types.ts`)

```
export interface ProductDto {
  id: number; name: string; price: number; quantity: number;
  gstPercentage: number; isActive: boolean; createdAt: string;
}
export interface CreateProductDto {
  name: string; price: number; quantity: number; gstPercentage: number;
}
```

Step 2 — Service

```
export const productService = {
  getAll: (params?) =>
    api.get<ApiResponse<PaginatedResponse<ProductDto>>>("/products", { params }).then(r => r.data.data),
  create: (dto: CreateProductDto) => api.post<ApiResponse<ProductDto>>("/products",
    dto).then(r => r.data.data),
  update: (id: number, dto: Partial<CreateProductDto>) =>
    api.patch(`/products/${id}`, dto).then(r => r.data.data),
  softDelete: (id: number) => api.delete(`/products/${id}`).then(r => r.data),
};
```

Step 3 — Hooks

```
export function useProducts(filters = {}) {
  return useQuery({ queryKey: QUERY_KEYS.products.list(filters), queryFn: () =>
    productService.getAll(filters) });
}

export function useCreateProduct() {
```

```

const qc = useQueryClient();
return useMutation({
  mutationFn: productService.create,
  onSuccess: () => { qc.invalidateQueries({ queryKey: QUERY_KEYS.products.lists() }); toast.success("Product created"); },
  onError: (e) => toast.error(e.response?.data?.message ?? "Failed to create"),
});
}

```

Step 4 — Page

```

export function ProductListPage() {
  const [page, setPage] = useState(1);
  const [search, setSearch] = useState("");
  const debouncedSearch = useDebounce(search, 400);
  const [isCreateOpen, setIsCreateOpen] = useState(false);

  const { data, isLoading } = useProducts({ page, limit: 20, search: debouncedSearch });
  const { mutate: createProduct, isPending } = useCreateProduct();

  return (
    <div className="p-6">
      <PageHeader title="Products" description="Manage your product inventory">
        <button onClick={() => setIsCreateOpen(true)} className="btn-primary">+ Add Product</button>
      </PageHeader>
      <input value={search} onChange={e => setSearch(e.target.value)} placeholder="Search products..." className="input mb-4" />
      <DataTable data={data?.data ?? []} columns={productColumns} isLoading={isLoading} meta={data?.meta} onPageChange={setPage} />
      <Dialog open={isCreateOpen} onOpenChange={setIsCreateOpen}>
        <DialogContent>
          <ProductForm onSubmit={d => createProduct(d, { onSuccess: () => setIsCreateOpen(false) }} isPending={isPending} />
        </DialogContent>
      </Dialog>
    </div>
  );
}

```

14. Performance Optimization

ThunderERP handles large datasets — thousands of products, orders, and customers. Performance must be considered from the start, not added as an afterthought.

14.1 Component Memoization

```
// React.memo — prevents re-render when props have not changed:
const ProductCard = React.memo(function ProductCard({ product }: Props) {
  return <div>{product.name}</div>;
});

// When to use React.memo:
// 1. Component renders frequently (in a list)
// 2. Props rarely change
// 3. Component is computationally expensive to render

// When NOT to use it:
// Simple components — the overhead of comparison exceeds the saving.
// Components that always re-render anyway (every render, new props).
```

14.2 Virtualization for Large Lists

If a table has more than 100 rows visible at once, use virtualization. It renders only the visible rows, not all rows.

```
npm install @tanstack/react-virtual

import { useVirtualizer } from "@tanstack/react-virtual";

const parentRef = useRef<HTMLDivElement>(null);

const virtualizer = useVirtualizer({
  count: products.length,
  getScrollElement: () => parentRef.current,
  estimateSize: () => 52, // Row height in px
});

return (
  <div ref={parentRef} style={{ height: "600px", overflow: "auto" }}>
    <div style={{ height: virtualizer.getTotalSize() }}>
      {virtualizer.getVirtualItems().map(virtualRow => (
        <div key={virtualRow.key}
          style={{ position: "absolute", top: virtualRow.start, height:
virtualRow.size }}
          className="w-full">
          <ProductRow product={products[virtualRow.index]} />
        </div>
      ))}
    </div>
  </div>
```

```
</div>  
);
```

14.3 Code Splitting & Lazy Loading

```
// Lazy load pages — they only download when the user navigates to them:  
import { lazy, Suspense } from "react";  
  
const ProductListPage = lazy(() =>  
import("./modules/inventory/pages/ProductListPage"));  
const OrderListPage   = lazy(() => import("./modules/sales/pages/OrderListPage"));  
const InvoiceListPage  = lazy(() =>  
import("./modules/finance/pages/InvoiceListPage"));  
  
// Wrap the router in Suspense:  
<Suspense fallback={<PageSkeleton />}>  
  <RouterProvider router={router} />  
</Suspense>  
  
// This splits each module into a separate JS bundle.  
// Initial load is faster — user only downloads code for the current module.
```

14.4 Performance Checklist

Issue	Solution
Whole page re-renders on every keystroke	Move state down. Only the input component should re-render.
Expensive list filtering	Wrap with useMemo. Recompute only when data or filter changes.
Too many API calls on mount	Check useEffect dependencies. Use TanStack Query — it deduplicates calls.
Slow initial load	Add lazy loading for all page-level components.
Re-creating objects/arrays on every render	Move to useMemo or outside the component if static.
Large bundle size	npm run build && npx vite-bundle-visualizer — identify large dependencies.

15. Testing Strategy

Tests in ThunderERP are not optional. At minimum, every critical path that involves user interactions with money or stock must be tested. Write tests as you build, not after.

15.1 Testing Tools

Tool	Purpose
Vitest	Test runner — fast, Vite-native. Replaces Jest.
React Testing Library	Tests components from the user's perspective (clicks, text, forms).
MSW (Mock Service Worker)	Intercepts HTTP calls in tests — no real API needed.
@testing-library/user-event	Simulates real user interactions (typing, clicking).

15.2 What to Test

- Component rendering — does it render without crashing with required props?
- User interactions — form submission, button clicks, input changes.
- Loading and error states — does the skeleton show? Does the error message appear?
- Custom hooks — does useProducts return the right data? Does useDebounce delay correctly?
- Auth flow — does ProtectedRoute redirect to /login when no token?
- Critical ERP paths — order creation form, stock adjustment, invoice payment flow.

15.3 Example Tests

```
// src/modules/inventory/components/__tests__/ProductForm.test.tsx
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { ProductForm } from "../ProductForm";

describe("ProductForm", () => {
  it("renders all required fields", () => {
    render(<ProductForm onSubmit={() => {}} />);
    expect(screen.getByPlaceholderText("e.g. Widget Pro")).toBeInTheDocument();
  });

  it("shows validation error when name is too short", async () => {
    render(<ProductForm onSubmit={() => {}} />);
    await userEvent.type(screen.getByPlaceholderText("e.g. Widget Pro"), "A");
    await userEvent.click(screen.getByText("Save Product"));
    expect(await screen.findByText("Name must be at least 2
characters")).toBeInTheDocument();
  });

  it("calls onSubmit with correct data when valid", async () => {
    const mockSubmit = vi.fn();
  });
});
```

```
render(<ProductForm onSubmit={mockSubmit} />);
await userEvent.type(screen.getByPlaceholderText("e.g. Widget Pro"), "Widget
Pro");
// ... fill other required fields
await userEvent.click(screen.getByText("Save Product"));
expect(mockSubmit).toHaveBeenCalledWith(expect.objectContaining({ name: "Widget
Pro" }));
});
});
```

15.4 MSW — Mock API in Tests

```
// src/mocks/handlers.ts
import { http, HttpResponse } from "msw";

export const handlers = [
  http.get("http://localhost:3000/products", () => {
    return HttpResponse.json({
      success: true,
      data: { data: [{ id: 1, name: "Widget Pro", price: 499, quantity: 100 }],
meta: { total: 1 } }
    });
  }),
];

// src/mocks/server.ts
import { setupServer } from "msw/node";
export const server = setupServer(...handlers);

// vitest.setup.ts:
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

16. CI/CD & Production Practices

Every line of code you write will go through a build pipeline before reaching users. Understanding this process is part of being a professional developer.

16.1 Pre-Commit Checks

```
# Install lint-staged + husky to run checks before every commit:
npm install -D husky lint-staged
npx husky init

# .husky/pre-commit:
npx lint-staged

# package.json:
"lint-staged": {
  "*.ts,tsx": ["eslint --fix", "prettier --write"]
}
```

16.2 Build Commands

Command	Purpose & When to Run
npm run dev	Start development server with HMR. Use during development.
npm run build	Compile TypeScript + bundle for production. Must pass before any PR merge.
npm run preview	Preview the production build locally before deploying.
tsc --noEmit	Type-check without generating files. Run in CI pipeline.
npm run lint	Run ESLint. Must pass with zero errors before PR.
npm run test	Run all tests with Vitest.
npm run test:coverage	Run tests + generate coverage report.

 **GOLDEN CI RULE:** Never merge code that fails "npm run build". A build failure means TypeScript errors or import problems that WILL cause a blank screen in production. Zero tolerance.

16.3 Deployment

```
# Build the production bundle:
npm run build
# Output: dist/ folder — this is what gets deployed

# Deploy to Vercel (easiest):
npm install -g vercel
vercel --prod
```

```
# Deploy to Netlify:
# Build command: npm run build
# Publish directory: dist

# Environment variables in hosting platform:
# VITE_API_URL = https://api.your-production-domain.com
```

17. Debugging Playbook

When something breaks, do not guess. Follow this systematic approach.

17.1 Debugging by Symptom

Symptom	Diagnosis Steps
UI shows empty / no data	1. Open Network tab — did the API request fire? 2. Check the response status code. 3. Check the response body matches the type you expect. 4. Check queryKey — is the data cached under a different key?
API request not firing	1. Check the event handler — is onClick/onSubmit wired correctly? 2. Check useEffect dependencies. 3. Check if the component is actually mounted. 4. Check if a loading/disabled state is blocking it.
Wrong data displayed	1. Check Network tab → response body. 2. Compare with Swagger contract. 3. Check the TypeScript type — does it match reality? 4. Check if you are reading from the right query key.
Infinite re-render loop	1. Check useEffect dependencies for objects/functions (recreated each render). 2. Move static values outside component. 3. Wrap with useMemo/useCallback.
Form not submitting	1. Check Zod validation — open browser console for zod errors. 2. Check if the form has a submit button with type="submit". 3. Check handleSubmit is on the form element, not the button.
White screen / blank page	1. Open browser console — usually a JavaScript error. 2. Check for import errors. 3. Run npm run build — type errors appear here.
Stale data after mutation	1. Check queryClient.invalidateQueries — is it called with the right key? 2. Check the key matches what useQuery is using. 3. Open React Query DevTools and inspect the cache.

17.2 Essential Developer Tools

Tool	How to Use
Browser DevTools — Network tab	See every HTTP request, its payload, and response. First stop for API debugging.
Browser DevTools — Console	JavaScript errors, console.log output. Check here when UI shows nothing.
React DevTools (browser extension)	Inspect component tree, props, and state. Find which component holds stale data.
TanStack Query DevTools	Inspect query cache, see which queries are loading/stale/fresh. Built into dev setup.
Swagger UI (http://localhost:3000/api)	Test API calls directly. Verify response shapes before building the frontend.

18. Frontend–Backend Contract Rules

The frontend and backend are developed in parallel. Without strict contract rules, one side changes a field name and the other breaks silently. These rules prevent that.

18.1 The Contract

SOURCE OF TRUTH: The NestJS backend Swagger page at <http://localhost:3000/api> is the single source of truth for all API contracts. Always check it before building a frontend feature.

Contract Rule	Implementation
Never hardcode API URLs in components	All URLs live in the service layer. Base URL from <code>import.meta.env.VITE_API_URL</code> .
Never assume response shape	Check Swagger. Type every API response. If the shape changes, TypeScript will catch it.
DTO field names must match exactly	If backend sends "totalAmount", frontend type must use "totalAmount" — not "total".
Error message format is standardized	Backend sends { statusCode, message, timestamp }. Frontend reads <code>error.response.data.message</code> .
Pagination params are standardized	Always: page (1-indexed), limit. Response always: <code>data[], meta { total, page, totalPages }</code> .
Date format is always ISO string	Dates come as "2024-01-15T10:30:00.000Z". Use <code>formatDate()</code> from <code>shared/utils/formatters</code> .

18.2 Syncing Types with Backend

```
// Option 1 (current): Manual sync
// When backend adds a new field, manually update the TypeScript type.
// Use Swagger as the reference.

// Option 2 (future): Auto-generate types from Swagger
// npm install -D openapi-typescript
// npx openapi-typescript http://localhost:3000/api-json -o
// src/types/api.generated.ts

// This generates TypeScript types automatically from the Swagger JSON.
// Run as part of your development workflow when backend schema changes.
```

19. SRS Module Mapping

Each frontend module maps to a backend module. Here is the complete mapping of screens, components, and API endpoints for each ThunderERP module.

19.1 Module Screen Map

Module	Screens	Key Components
Auth	LoginPage, ForgotPasswordPage	LoginForm, AuthLayout
Dashboard	DashboardPage	KpiCard, RevenueChart, LowStockAlert, RecentOrdersTable
Inventory	ProductListPage, ProductDetailPage	ProductTable, ProductForm, StockBadge, StockAdjustmentModal
Sales	OrderListPage, OrderDetailPage, CreateOrderPage	OrderTable, OrderItemsForm, OrderStatusStepper, OrderSummaryCard
CRM	LeadListPage, CustomerListPage, CustomerDetailPage	LeadPipeline, LeadForm, CustomerCard, OrderHistoryTable
Finance	InvoiceListPage, InvoiceDetailPage, PaymentModal	InvoiceTable, InvoiceStatusBadge, PaymentForm, LedgerView

19.2 Dashboard KPI Cards

```
// The dashboard fetches aggregate data - these APIs should return pre-calculated values
// Backend provides: GET /dashboard/summary

interface DashboardSummary {
  todayRevenue:      number;
  pendingOrders:      number;
  lowStockProducts:  number;
  unpaidInvoices:     number;
  revenueThisMonth:  number;
  ordersThisMonth:    number;
}

// KPI Card component:
function KpiCard({ title, value, unit, trend }: KpiCardProps) {
  return (
    <div className="border rounded-lg p-4 bg-white shadow-sm">
      <p className="text-sm text-gray-500">{title}</p>
      <p className="text-2xl font-bold text-primary mt-1">
        {unit === "currency" ? formatCurrency(value) : value}
      </p>
      {trend && <TrendIndicator value={trend} />}
    </div>
  );
}
```

| }

20. Golden Rules of ThunderERP Frontend

RULE 1: Business logic belongs in NestJS Services — NEVER in React components. The frontend never calculates tax, validates stock, or applies discounts. It only displays results.

RULE 2: Never use TypeScript "any". Define proper interfaces. If you do not know the shape, check Swagger. "any" hides bugs until they reach production.

RULE 3: Never call Axios directly in components. ALL API calls go through the service layer.

RULE 4: Use TanStack Query for all server data. Never useState + useEffect for fetching. TanStack Query handles caching, loading, errors, and refetching.

RULE 5: Handle all three states of every API call: loading (skeleton), error (message + retry), success (data). Never leave a state unhandled.

RULE 6: Components must stay small — under 150 lines. If a component grows larger, extract a sub-component or a custom hook.

RULE 7: Never hardcode API URLs or string literals. All routes in shared/constants/routes.ts. All query keys in shared/constants/queryKeys.ts.

RULE 8: npm run build must pass before any PR merge. Type errors and import issues show up here. A failed build = blank screen in production.

21. Common Mistakes — Never Do These

Mistake	Why It Matters & Correct Approach
Business logic inside components	Impossible to test, reuse, or maintain. Move all conditional calculations to hooks or services.
Calling Axios directly in useEffect	No caching, no deduplication, no loading state, memory leaks on unmount. Use TanStack Query.
Using "any" for TypeScript types	"any" disables TypeScript entirely. Define proper interfaces from Swagger.
Duplicating backend validation	Frontend validation is UX only. Backend always validates too. Never trust only frontend.
Not handling loading states	Users see empty or broken UI on slow connections. Always show a skeleton.
Not handling error states	API failures are invisible to the user. Always show a message and retry option.
Hardcoding API URLs in components	When the API URL changes, 20 files need updates. Use the Axios instance.
Mixing module logic across folders	Inventory code in the sales folder creates chaos. Strict separation by module.
Pushing code that fails npm run build	TypeScript errors in your code become runtime errors in production.
Reading env variables without VITE_ prefix	Vite only exposes env vars prefixed with VITE_ to the browser. Others are undefined.
Directly mutating state	setState(arr.push(x)) is a bug. Always return a new object/array.
Missing queryKey dependencies	Filters/pagination params not in the queryKey means stale data is shown for different filters.

22. Where Frontend Breaks — The Failure Chain

Wrong API endpoint URL

- Request returns 404
- UI shows empty state forever
- User thinks there is no data

No error handling on API call

- Network error throws unhandled exception
- React component crashes (white screen)
- User loses all unsaved work

Missing queryKey dependency (e.g., page number)

- Page 2 shows the same data as Page 1
- User thinks pagination is broken

TypeScript "any" on API response

- Backend adds new field / renames field
- Frontend silently shows undefined
- Bug goes unnoticed until demo

Business logic in the frontend (e.g., discount calculation)

- Frontend and backend calculate differently
- Invoice total does not match order total
- Financial data is inconsistent

No loading state

- User clicks button twice before response arrives
- Duplicate orders/records created

23. Learning Roadmap

Days 1–3 — Foundation

- Complete full environment setup (Section 2).
- Create the project, run dev server, see it in the browser.
- Read the Golden Frontend Flow (Section 1.2) until you can recite it.
- Build the Login page (Section 8.3) — your first real feature.

Week 1 — Core React

- Master all core hooks: useState, useEffect, useMemo, useCallback (Section 5).
- Set up the folder structure (Section 3).
- Configure TanStack Query and Axios (Sections 6.1–6.4).
- Build a basic product list page with real API data and loading/error states.
- Add routing and protect routes with ProtectedRoute (Section 7).

Week 2 — Forms & Data

- Master React Hook Form + Zod (Section 9).
- Build the complete Inventory module: list, create, edit, soft delete (Section 13).
- Implement the DataTable component with pagination (Section 10.1).
- Set up status badges and formatters (Sections 10.2, 10.5).

Week 3 — State & Patterns

- Master TanStack Query mutations with cache invalidation (Section 11.2).
- Implement Zustand auth store (Section 8.2).
- Build modal create/edit flows (Section 10.3).
- Add toast notifications and error boundaries (Section 12).
- Write your first component tests (Section 15).

Week 4 — Advanced

- Build the Sales Orders module with status workflow.
- Implement the Dashboard with KPI cards.
- Add code splitting and lazy loading (Section 14.3).
- Set up CI/CD pipeline awareness (Section 16).

Month 2+ — Professional Level

- Performance optimization — React DevTools Profiler, memoization strategy.
- Full test coverage — MSW, Testing Library, Vitest.
- Accessibility — ARIA roles, keyboard navigation, screen reader support.
- Full system integration — frontend + backend + database working together.

24. Mandatory Screens — Build These First

These six screens are the core of ThunderERP. Build them in this order. Each one builds on the skills from the previous.

Screen	What It Teaches
1. Login Page	Forms, Zod validation, Zustand auth store, navigation.
2. Dashboard	TanStack Query, multiple API calls, KPI card components, charts.
3. Product List + CRUD	DataTable, pagination, create/edit modals, mutations, cache invalidation.
4. Order List + Status Workflow	State machine UI, status badge, detail page, nested data.
5. Invoice View + PDF Preview	Complex data display, computed totals, payment recording modal.
6. Customer 360 View	Aggregated data from multiple endpoints, order history, CRM data.

25. Final Mindset — What You Are Really Building

Frontend development in ThunderERP is NOT just styling. It is not making things look pretty. It is engineering the human interface of a mission-critical enterprise system that handles money, inventory, and customer relationships.

What the Frontend Really Is

DATA FLOW ENGINEERING: You are building the layer that moves data from a database to a human brain and back again. Every decision about state, caching, and API integration is a data engineering decision.

USER INTERACTION CONTROL: You are controlling what actions the user can take, in what order, and with what constraints. The order creation flow, the status workflow, the form validation — these are user experience decisions with real business consequences.

THE FACE OF COMPLEXITY: The backend has transactions, state machines, and complex business rules. Your job is to make all of that invisible — to make the most complex operations feel simple and intuitive. That is hard engineering.

The Professional Standard

A senior frontend engineer writes code that a junior can read and understand six months later. Small components, clear names, extracted hooks, no clever tricks. The goal is boring, predictable, maintainable code.

You are not learning React. You are learning how to build interfaces for enterprise software. React is just the tool.

Build screens that make complex systems feel simple.